

Reviewer Pack — Minimal Hodge Decomposition Complexity and DPLL Proof Tree H...

Entry 86150a483966 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 14:12:03 UTC

Minimal Hodge Decomposition Complexity and DPLL Proof Tree Height Inequality

Entry ID: 86150a483966 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 14:12:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory (DPLL Search Trees)

Statement:

For every Boolean satisfiability problem (SAT) instance φ , the minimal Hodge decomposition complexity ($\text{hdc}(\varphi)$) of φ is linearly correlated with its DPLL proof tree height $h(\varphi)$, such that $\text{hdc}(\varphi) = \Theta(h(\varphi))$.

Rationale (proposer's reasoning):

The Hodge decomposition in algebraic geometry provides a way to understand the structure of complex varieties. If the minimal Hodge decomposition complexity can be shown to be related to the DPLL proof tree height, it may reveal a deep connection between geometric structures and computational complexity, potentially leading to new insights into NP-completeness.

Taxonomy category: hodge_decomposition_complexity_dpll_proof_tree_height (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c9aac4489f87c74f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all SAT instances φ with n variables, $\text{hdc}(\varphi)$ falls within a linear relationship with $h(\varphi)$, where $\text{hdc}(\varphi) = \Theta(h(\varphi))$, and this relationship holds true for at least 80% of the generated instances. The criterion is falsified if any instance violates this relationship or if less than 70% of the instances show this correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "DPLL proof tree height" - "minimal Hodge decomposition complexity" AND "Boolean satisfiability problem" - "Algebraic Geometry" AND "Complexity Theory"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot in column i
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j

        # Swap rows
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate entries below the pivot
        factor = Fraction(A[i][i])
        for j in range(i+1, n):
            A[j][i] /= factor

    return A

def hodge_complexity(phi):
    # Convert phi to a matrix representation (simplified example)
    n = len(phi)
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if phi[i][j]:
                A[i][j] = 1
```

```

rank = 0
for row in gaussian_elimination(A):
    if any(row):
        rank += 1

return rank

def dpll_proof_tree_height(phi):
    # Simplified DPLL algorithm to estimate proof tree height (simplified example)
    n = len(phi)
    clauses = phi[:]
    stack = []

    def backtrack():
        while True:
            if not stack:
                return len(clauses) # Return the current depth

            clause = next((c for c in clauses if any(lit in c for lit in stack)), None)
            if not clause:
                stack.pop()
                continue

            literal = next(lit for lit in clause if lit in stack)
            if literal > 0:
                stack.append(-literal)
            else:
                stack.append(-literal)

    return backtrack()

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    hdc_values = []
    h_values = []

    for n in n_values:
        phi = [[random.choice([True, False]) for _ in range(n)] for _ in range(n)]

        hdc_value = hodge_complexity(phi)
        h_value = dpll_proof_tree_height(phi)

        hdc_values.append(hdc_value)
        h_values.append(h_value)

    mean_hdc = sum(hdc_values) / len(hdc_values)
    mean_h = sum(h_values) / len(h_values)
    std_dev = (sum((x - mean_hdc)**2 for x in hdc_values) / len(hdc_values))**0.5

    correlation_coefficient = sum((hdc_values[i] - mean_hdc) * (h_values[i] - mean_h) for i in range(n))

    if correlation_coefficient >= 0.8:
        conjecture_holds = True
        counterexample = ""

```

```

else:
    conjecture_holds = False
    counterexample = "correlation_coefficient < 0.8"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(hdc_values),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient statistical support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2bc12f3.py", line 120, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2bc12f3.py", line 86, in run_trial
    hdc_value = hodge_complexity(phi)
                ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2bc12f3.py", line 47, in hodge_complexity
    for row in gaussian_elimination(A):
               ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2bc12f3.py", line 33, in gaussian_elimination
    A[j][i] /= factor

```

```

File "/usr/lib/python3.12/fractions.py", line 630, in reverse

```

```

return monomorphic_operator(Fraction(a), b)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/fractions.py", line 763, in _div
    raise ZeroDivisionError('Fraction(%s, 0)' % db)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify the conjecture's support or falsification based on the pre-registered criteria. | next: Re-run the test without errors to collect data and evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17968
2	preregistration	ollama_remote	glm4:latest	0	0	10422
3	novelty	ollama_remote	glm4:latest	0	0	8395
4	novelty	ollama_remote	glm4:latest	0	0	9573
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27345
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15958
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12037
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13327
9	judge	ollama_remote	glm4:latest	0	0	16654

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131678 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/86150a483966.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/86150a483966.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/86150a483966.tar.gz* (if generated)